

Unidad IV: Implementaciones dinámicas y eficiencia

Clase 7 – Pensar en costos: eficiencia temporal y espacial

Objetivo

Desarrollar pensamiento analítico para justificar decisiones de diseño.

Tema

- Análisis de complejidad.
- Orden de magnitud.
- Criterios de selección de implementación.

Pensar en costos: eficiencia temporal y espacial

En las clases anteriores se trabajó con distintos Tipos de Datos Abstractos: pila, cola, cola con prioridad, conjunto, diccionario simple y diccionario múltiple. También se analizaron distintas formas de implementarlos, primero mediante estructuras estáticas basadas en arreglos y luego mediante estructuras dinámicas basadas en nodos enlazados.

Esta clase tiene como propósito ordenar ese recorrido y responder una pregunta central: **si el TDA es el mismo, por qué cambia el costo de sus operaciones.**

La respuesta está en la diferencia entre especificación e implementación. La especificación de un TDA indica qué operaciones existen y qué comportamiento deben cumplir. Por ejemplo, una pila debe permitir apilar, desapilar, consultar el tope y saber si está vacía. Sin embargo, la especificación no dice si la pila se guarda en un arreglo, si el tope está al inicio, si el tope está al final o si se implementa mediante nodos enlazados.

La implementación, en cambio, define cómo se almacenan los datos y cómo se resuelve internamente cada operación. Por eso, dos implementaciones pueden respetar exactamente el mismo TDA y, aun así, tener costos distintos. Esta idea es central en el apunte de cátedra, donde primero se presentan los TDAs como modelos abstractos y luego se estudian distintas implementaciones estáticas y dinámicas para esos mismos comportamientos.

El análisis de costos permite comparar esas implementaciones usando criterios simples. En el enfoque de la materia se trabaja principalmente con tres categorías: costo constante, costo lineal y costo polinómico. El costo constante, indicado como **C**, representa operaciones cuyo tiempo no depende de la cantidad de elementos almacenados. El costo lineal, indicado como **L**, aparece cuando la operación debe recorrer o desplazar elementos. El costo polinómico, indicado como **P**, aparece cuando

el crecimiento es mayor al lineal, por ejemplo cuando una operación lineal se ejecuta repetidamente dentro de otro recorrido.

Además del tiempo, también interesa el costo espacial. Las implementaciones estáticas utilizan arreglos de tamaño fijo. Esto puede ser simple y eficiente, pero exige reservar previamente una capacidad, aun cuando no siempre se use completa. Las implementaciones dinámicas utilizan nodos enlazados. Esto permite crecer según la necesidad, pero cada nodo requiere memoria adicional para guardar referencias. Por eso, una estructura dinámica no debe entenderse automáticamente como “mejor”, sino como una alternativa que modifica ciertos costos y ofrece mayor flexibilidad.

1. PilaTDA: comparación de implementaciones

El TDA Pila representa una estructura donde el último elemento que ingresa es el primero que sale. Sus operaciones básicas son InicializarPila, Apilar, Desapilar, Tope y PilaVacía.

En la cátedra se trabajan tres formas relevantes de implementación: pila con tope al final, pila con tope al inicio y pila dinámica con nodos.

En la **PilaTF**, o pila con tope al final, los elementos se guardan en un arreglo y se utiliza una variable índice para indicar la cantidad de elementos cargados. El nuevo elemento se agrega en la posición indicada por índice, y luego índice aumenta. Para desapilar, simplemente se disminuye índice. No hay corrimientos.

Por eso, en esta implementación, Apilar es constante, Desapilar es constante, Tope es constante y PilaVacía también es constante.

En la **PilaTI**, o pila con tope al inicio, el tope se mantiene en la posición cero del arreglo. Esto permite consultar el tope rápidamente, pero tiene una consecuencia importante: al apilar un nuevo elemento, todos los elementos existentes deben correrse hacia la derecha para dejar libre la posición cero. Al desapilar, los elementos restantes deben correrse hacia la izquierda. Por eso, Apilar y Desapilar son lineales.

En la **PilaLD**, implementación dinámica con nodos enlazados, el tope de la pila es el primer nodo de la lista. Para apilar se crea un nuevo nodo, se lo enlaza con el nodo que antes era el primero y luego se actualiza la referencia al nuevo tope. Para desapilar, se cambia la referencia del primero al siguiente nodo. No hay corrimiento.

La comparación queda así:

Operación	PilaTI	PilaTF	PilaLD
InicializarPila	C	C	C
Apilar	L	C	C
Desapilar	L	C	C
Tope	C	C	C

Operación	PilaTI	PilaTF	PilaLD
PilaVacía	C	C	C

La conclusión importante es que la pila dinámica mejora claramente frente a la pila con tope al inicio, porque evita corrimientos. Sin embargo, respecto de la pila con tope al final, no mejora el costo temporal de las operaciones básicas, ya que ambas logran costo constante. La diferencia principal pasa por el uso de memoria: la estática usa un arreglo de capacidad fija; la dinámica crea nodos a medida que se necesitan.

2. ColaTDA: comparación de implementaciones

El TDA Cola representa una estructura donde el primer elemento que ingresa es el primero que sale. Sus operaciones básicas son InicializarCola, Acolar, Desacolar, Primero y ColaVacía.

En la cátedra se presentan dos implementaciones estáticas principales y una dinámica.

En la **ColaPU**, el último elemento ingresado queda en la posición inicial del arreglo. Para acolar un nuevo elemento en la posición cero, se deben correr los elementos existentes hacia la derecha. En cambio, para desacolar se elimina el primero ingresado, que queda ubicado hacia el final de la zona ocupada, por lo que alcanza con disminuir el índice. En esta versión, Acolar es lineal y Desacolar es constante.

En la **ColaPI**, el primer elemento ingresado queda en la posición inicial del arreglo. Para acolar, el nuevo elemento se agrega al final de la zona ocupada, por lo que Acolar es constante. Pero para desacolar, se elimina el elemento de la posición cero y luego hay que correr todos los elementos hacia la izquierda. Por eso, Desacolar es lineal.

En la **ColaLD**, implementación dinámica, se mantienen dos referencias: una al primer nodo y otra al último nodo. Para acolar se agrega un nodo al final. Para desacolar se avanza la referencia del primero al siguiente nodo. Si la cola queda vacía, también se actualiza la referencia al último. Al tener ambas referencias, no se necesita recorrer ni desplazar elementos.

La comparación queda así:

Operación	ColaPU	ColaPI	ColaLD
InicializarCola	C	C	C
Acolar	L	C	C
Desacolar	C	L	C
Primero	C	C	C
ColaVacía	C	C	C

Este es uno de los mejores ejemplos para la clase. En las implementaciones estáticas vistas por la cátedra, siempre hay una operación principal que queda penalizada por

corrimiento: o Acolar o Desacolar. En la implementación dinámica, ambas operaciones son constantes porque se resuelven modificando referencias.

Por eso, si el problema requiere muchas altas y bajas en una cola, la ColaLD resulta más eficiente temporalmente que las versiones estáticas vistas. La diferencia no está en el TDA Cola, sino en la forma de implementarlo.

3. ColaPrioridadTDA: comparación de implementaciones

El TDA Cola con Prioridad representa una cola donde cada elemento ingresa asociado a una prioridad. Al consultar o eliminar, se accede al elemento de mayor prioridad. Si dos elementos tienen la misma prioridad, se respeta el orden de llegada.

Sus operaciones son InicializarCola, AcolarPrioridad, Desacolar, Primero, ColaVacia y Prioridad.

La cátedra presenta tres implementaciones principales.

La primera es **ColaPrioridadDA**, con dos arreglos paralelos: uno para los elementos y otro para las prioridades. Ambos arreglos deben mantenerse sincronizados. Al acolar con prioridad, se busca la posición correcta y se desplazan elementos y prioridades. Como se deben mover datos en ambos arreglos, el costo sigue siendo lineal.

La segunda es **ColaPrioridadAO**, con un arreglo de objetos o elementos compuestos. Cada posición del arreglo contiene un objeto con valor y prioridad. Conceptualmente es más ordenado que usar dos arreglos paralelos, porque el valor y su prioridad viajan juntos. Sin embargo, para insertar en la posición correcta también se deben desplazar elementos. Por eso, AcolarPrioridad sigue siendo lineal.

La tercera es **ColaPrioridadLD**, implementación dinámica con nodos que contienen valor, prioridad y referencia al siguiente nodo. La lista se mantiene ordenada por prioridad. Para insertar un nuevo nodo, hay que recorrer hasta encontrar la posición adecuada. Una vez encontrada, se modifican referencias. Esto evita corrimientos de arreglo, pero no evita la búsqueda de la posición.

La comparación queda así:

Operación	ColaPrioridadDA	ColaPrioridadAO	ColaPrioridadLD
InicializarCola	C	C	C
AcolarPrioridad	L	L	L
Desacolar	C	C	C
Primero	C	C	C
ColaVacia	C	C	C
Prioridad	C	C	C

La conclusión es muy importante: la implementación dinámica no siempre cambia el orden de magnitud. En la cola con prioridad, AcolarPrioridad sigue siendo lineal, porque el problema central no es el corrimiento sino la necesidad de encontrar la posición correcta según la prioridad.

Sin embargo, hay una diferencia práctica: en la versión estática se desplazan elementos del arreglo; en la versión dinámica se recorren nodos y luego se cambian referencias. Desde el punto de vista del análisis simplificado de la materia, ambas son lineales. Desde el punto de vista de diseño, la dinámica evita capacidad fija y corrimientos, pero incorpora nodos y referencias.

4. ConjuntoTDA: comparación de implementaciones

El TDA Conjunto permite almacenar elementos sin repetidos y sin orden. Sus operaciones básicas son InicializarConjunto, Agregar, Sacar, Elegir, Pertenece y ConjuntoVacio.

En la implementación estática **ConjuntoA**, los elementos se guardan en un arreglo y una variable indica cuántos elementos hay. Como el conjunto no mantiene orden, al eliminar un elemento no hace falta desplazar todos los demás. Se puede reemplazar el elemento eliminado por el último elemento cargado y disminuir la cantidad.

Sin embargo, para saber si un elemento pertenece al conjunto, hay que recorrer. Para agregar, antes hay que verificar si el elemento ya existe, porque el conjunto no admite repetidos. Por eso, Agregar es lineal. Para sacar, también hay que buscar el elemento, por lo que Sacar es lineal. Pertenece es lineal. En cambio, Elegir puede ser constante, porque se devuelve un elemento cualquiera.

En la implementación dinámica **ConjuntoLD**, los elementos se guardan en nodos enlazados. Como tampoco hay orden, se puede agregar al inicio de la lista. Pero antes de agregar hay que verificar que el elemento no exista. Esa verificación requiere recorrer la lista. Por eso, Agregar sigue siendo lineal. Sacar también es lineal porque hay que encontrar el nodo a eliminar. Pertenece es lineal porque recorre hasta encontrar o hasta llegar al final. Elegir es constante si se devuelve el primer nodo.

La comparación queda así:

Operación	ConjuntoA	ConjuntoLD
InicializarConjunto	C	C
Agregar	L	L
Sacar	L	L
Pertenece	L	L
Elegir	C	C
ConjuntoVacio	C	C

La conclusión es que pasar de arreglo a lista dinámica no mejora el costo temporal principal del conjunto. La razón es que el problema central es la búsqueda. Mientras los elementos no estén ordenados ni exista una estructura de acceso más eficiente, buscar seguirá siendo lineal.

La diferencia entre ambas implementaciones aparece más en el espacio y la flexibilidad. El conjunto estático depende de una capacidad fija. El conjunto dinámico crece mediante nodos, pero cada nodo requiere memoria adicional para la referencia.

5. DiccionarioSimpleTDA: comparación de implementaciones

El TDA Diccionario Simple permite asociar una clave con un único valor. Cada clave es única y tiene un solo valor asociado. Sus operaciones son InicializarDiccionario, Agregar, Eliminar, Recuperar y Claves.

En la implementación estática **DicSimpleA**, se utiliza un arreglo de elementos. Cada elemento contiene una clave y un valor. Para agregar una clave, primero se debe buscar si la clave ya existe. Si existe, se actualiza el valor. Si no existe, se agrega un nuevo elemento. Esa búsqueda de clave es lineal. Por eso, Agregar es lineal. Eliminar también debe buscar la clave, por lo que es lineal. Recuperar busca la clave para devolver el valor, entonces también es lineal. Claves debe recorrer todos los elementos para construir un conjunto con las claves, por lo que es lineal.

En la implementación dinámica **DicSimpleL**, se utiliza una lista de nodos de clave. Cada nodo contiene clave, valor y referencia al siguiente nodo. Para agregar, eliminar o recuperar, se debe recorrer la lista hasta encontrar la clave. Por eso, los costos principales siguen siendo lineales.

La comparación queda así:

Operación	DicSimpleA	DicSimpleL
InicializarDiccionario	C	C
Agregar	L	L
Eliminar	L	L
Recuperar	L	L
Claves	L	L

La conclusión es similar a la del conjunto: la implementación dinámica no cambia el orden de costo temporal, porque la búsqueda de clave sigue siendo secuencial. Cambia la forma de almacenar los datos, pero no aparece una estrategia de acceso más eficiente.

6. DiccionarioMultipleTDA: comparación de implementaciones

El TDA Diccionario Múltiple permite asociar una clave con varios valores. Una clave puede tener un conjunto de valores asociados, pero esos valores no deben repetirse.

Sus operaciones son InicializarDiccionario, Agregar, Eliminar, EliminarValor, Recuperar y Claves.

En la implementación estática **DicMultipleA**, se utiliza un arreglo de elementos. Cada elemento contiene una clave, un arreglo de valores y una cantidad de valores. Para agregar un par clave-valor, primero se busca la clave. Si no existe, se crea. Luego se busca si el valor ya está asociado a esa clave. Si no está, se agrega. En el análisis simplificado de la cátedra, esta operación se considera lineal.

Eliminar busca la clave y la elimina. EliminarValor busca la clave y luego busca el valor dentro de los valores asociados. Recuperar busca la clave y devuelve un conjunto con sus valores. Claves recorre las claves del diccionario. Todas estas operaciones son lineales en el enfoque de costos trabajado.

En la implementación dinámica **DicMultipleL**, se trabaja con una lista de nodos de clave y, para cada clave, una lista de nodos de valor. Para agregar, primero se busca el nodo de la clave. Luego se busca el valor dentro de la lista de valores de esa clave. Para eliminar un valor, ocurre algo similar. Si al eliminar el valor la clave queda sin valores asociados, también debe eliminarse la clave. Nuevamente, como las búsquedas son secuenciales, el costo es lineal.

La comparación queda así:

Operación	DicMultipleA	DicMultipleL
InicializarDiccionario	C	C
Agregar	L	L
Eliminar	L	L
EliminarValor	L	L
Recuperar	L	L
Claves	L	L

La conclusión es que el diccionario múltiple dinámico mejora en flexibilidad de memoria, porque no depende de un arreglo fijo de claves ni de arreglos fijos de valores. Pero desde el punto de vista temporal, al seguir usando búsquedas secuenciales, el costo continúa siendo lineal.

7. Comparación general entre estructuras estáticas y dinámicas

La comparación entre implementaciones estáticas y dinámicas no debe reducirse a decir que una es mejor que la otra. Cada una tiene ventajas y limitaciones.

Las implementaciones estáticas usan arreglos. Esto permite acceso directo por posición, estructuras simples y bajo costo adicional de memoria por elemento. Sin embargo, tienen capacidad fija y, según el criterio de almacenamiento, pueden requerir corrimientos costosos.

Las implementaciones dinámicas usan nodos enlazados. Permiten crecer a medida que se necesitan elementos y evitan muchos corrimientos. Sin embargo, cada nodo requiere memoria adicional para guardar referencias. Además, si la operación necesita buscar un elemento, la estructura dinámica no evita por sí sola el recorrido.

Por eso, la elección de implementación debe hacerse a partir de las operaciones dominantes. No se elige una estructura solo por su nombre, sino por el uso esperado.

Si se necesita una cola con muchas operaciones de entrada y salida, la implementación dinámica es muy conveniente porque Acolar y Desacolar son constantes.

Si se necesita una pila, una implementación estática con tope al final ya ofrece operaciones constantes. La dinámica también, pero no mejora el orden temporal; su ventaja puede estar en no depender de una capacidad fija.

Si se necesita un conjunto donde se consulta muchas veces pertenencia, tanto la versión estática no ordenada como la dinámica tienen costo lineal. En ese caso, el problema no se resuelve cambiando arreglo por lista, sino usando otra estrategia de organización que todavía no forma parte de este tramo inicial.

Si se necesita un diccionario simple o múltiple, la búsqueda de clave domina el costo. Mientras las claves estén almacenadas en forma secuencial, recuperar, agregar o eliminar seguirá siendo lineal.

Si se necesita una cola con prioridad, el costo fuerte está en insertar manteniendo el orden de prioridad. La dinámica evita corrimientos, pero no evita recorrer para encontrar la ubicación correcta.

8. Tabla integradora de costos

TDA	Implementación	Operaciones constantes principales	Operaciones lineales principales	Motivo del costo lineal
Pila	PilaTI	Tope, PilaVacía	Apilar, Desapilar	Corrimientos
Pila	PilaTF	Todas las básicas	Ninguna básica	Acceso directo al tope
Pila	PilaLD	Todas las básicas	Ninguna básica	Cambios de referencia
Cola	ColaPU	Desacolar, Primero, ColaVacía	Acolar	Corrimiento al insertar
Cola	ColaPI	Acolar, Primero, ColaVacía	Desacolar	Corrimiento al eliminar
Cola	ColaLD	Todas las básicas	Ninguna básica	Referencia a primero y último

TDA	Implementación	Operaciones constantes principales	Operaciones lineales principales	Motivo del costo lineal
Cola Prioridad	DA	Desacolar, Primero, Prioridad	AcolarPrioridad	Búsqueda/ubicación y corrimiento
Cola Prioridad	AO	Desacolar, Primero, Prioridad	AcolarPrioridad	Búsqueda/ubicación y corrimiento
Cola Prioridad	LD	Desacolar, Primero, Prioridad	AcolarPrioridad	Recorrido para ubicar prioridad
Conjunto	A	Elegir, ConjuntoVacio	Agregar, Sacar, Pertenece	Búsqueda secuencial
Conjunto	LD	Elegir, ConjuntoVacio	Agregar, Sacar, Pertenece	Búsqueda secuencial
Diccionario Simple	A	Inicializar	Agregar, Eliminar, Recuperar, Claves	Búsqueda de clave / recorrido
Diccionario Simple	L	Inicializar	Agregar, Eliminar, Recuperar, Claves	Búsqueda de clave / recorrido
Diccionario Múltiple	A	Inicializar	Agregar, Eliminar, EliminarValor, Recuperar, Claves	Búsqueda de clave y/o valor
Diccionario Múltiple	L	Inicializar	Agregar, Eliminar, EliminarValor, Recuperar, Claves	Búsqueda de clave y/o valor

9. Criterios para seleccionar una implementación

Una implementación debe seleccionarse considerando el contexto del problema. No alcanza con mirar una tabla de costos de manera aislada. Hay que preguntarse qué operación será más frecuente, cuántos elementos se esperan, si el tamaño máximo es conocido, si se necesita evitar desperdicio de memoria, si habrá muchas inserciones o eliminaciones, y si el costo principal está en recorrer, desplazar o modificar referencias.

Cuando el tamaño máximo es conocido y pequeño, una implementación estática puede ser suficiente. Puede ser más simple de programar y de entender.

Cuando el tamaño es variable o imprevisible, una implementación dinámica puede ser más adecuada porque permite crecer sin reservar previamente toda la capacidad.

Cuando la operación dominante requiere corrimientos en arreglo, una implementación dinámica puede mejorar significativamente el costo. Esto se ve claramente en la cola.

Cuando la operación dominante requiere buscar un elemento, pasar a dinámica no necesariamente mejora el costo temporal. Esto se ve en conjunto y diccionarios.

Cuando se necesita mantener un criterio de orden, como en cola con prioridad, hay que aceptar que insertar puede requerir recorrer. La implementación dinámica puede evitar corrimientos, pero no elimina la necesidad de encontrar la posición correcta.

COSTO ESPACIAL

El análisis de costos no solo puede realizarse desde el tiempo que tarda una operación, sino también desde la memoria que necesita una estructura o un algoritmo para funcionar. A este segundo aspecto se lo denomina **costo espacial** o **complejidad espacial**.

Mientras el costo temporal pregunta “cuántas operaciones se realizan cuando aumenta la cantidad de elementos”, el costo espacial pregunta “cuánta memoria se necesita cuando aumenta la cantidad de elementos”.

En esta materia, el análisis principal se realiza sobre el costo temporal. Sin embargo, la comparación entre implementaciones estáticas y dinámicas permite introducir el costo espacial de manera conceptual, porque ambas estrategias almacenan la información de forma diferente.

En una **implementación estática**, los datos se guardan generalmente en arreglos. El arreglo tiene una capacidad definida desde el momento de su creación. Por ejemplo, muchas implementaciones vistas en el apunte inicializan arreglos de tamaño 100. Esto significa que la memoria para esas posiciones queda reservada aunque la estructura tenga pocos elementos cargados. Si una pila tiene solo 5 elementos, pero su arreglo fue creado con 100 posiciones, el espacio disponible restante existe dentro de la estructura aunque no esté siendo utilizado por datos reales.

Por eso, en una estructura estática, el costo espacial depende principalmente de la **capacidad reservada**, no solo de la cantidad de elementos efectivamente cargados. Esta estrategia puede ser simple y eficiente cuando el tamaño máximo es conocido, pero puede desperdiciar memoria si se reserva mucho más de lo necesario. También puede quedar limitada si la cantidad de elementos supera la capacidad prevista.

En una **implementación dinámica**, los datos se guardan mediante nodos enlazados. Cada nodo se crea a medida que se necesita incorporar un nuevo elemento. El apunte explica que, a diferencia de los arreglos, las estructuras dinámicas reservan memoria según la necesidad y que en Java la memoria que deja de usarse queda disponible mediante el mecanismo de Garbage Collection. También señala que cada nodo contiene el valor almacenado y una referencia hacia otro nodo.

Esto significa que una estructura dinámica no reserva de entrada una gran cantidad de posiciones vacías. Si la estructura tiene 5 elementos, se crean 5 nodos. Si luego tiene

20 elementos, se crean más nodos. Esta es una ventaja cuando el tamaño de la estructura es variable o desconocido.

Sin embargo, la estructura dinámica también tiene un costo espacial adicional: cada nodo no guarda solamente el dato, sino también una o más referencias. En una pila o una cola dinámica simple, cada nodo guarda el valor y una referencia al siguiente nodo. En una cola con prioridad dinámica, cada nodo guarda el valor, la prioridad y la referencia al siguiente. En un diccionario múltiple dinámico, pueden existir nodos de clave y, asociados a cada clave, nodos de valores. Por eso, aunque la memoria se reserve de manera más flexible, cada elemento puede requerir más memoria que en un arreglo simple.

La comparación puede resumirse así:

Aspecto	Implementación estática	Implementación dinámica
Forma de almacenamiento	Arreglos	Nodos enlazados
Reserva de memoria	Se reserva una capacidad fija al inicializar	Se reserva memoria a medida que se agregan nodos
Riesgo principal	Desperdiciar espacio o quedarse sin capacidad	Usar memoria adicional por referencias
Ventaja principal	Estructura simple y compacta	Flexibilidad para crecer o reducirse
Relación con la cantidad real de elementos	Puede haber posiciones reservadas no usadas	Se crean nodos según los elementos necesarios

Aplicar costo espacial no significa contar exactamente los bytes ocupados por cada estructura. En esta etapa, alcanza con identificar qué memoria adicional necesita cada implementación y cómo crece cuando aumenta la cantidad de elementos.

Por ejemplo, una PilaTF estática usa un arreglo de enteros y una variable índice. Si el arreglo fue creado con 100 posiciones, ese espacio queda reservado desde el inicio. Una PilaLD dinámica, en cambio, usa un nodo por cada elemento. Si hay n elementos, hay n nodos; cada nodo contiene el valor y la referencia al siguiente. Temporalmente ambas pueden tener operaciones constantes, pero espacialmente organizan la memoria de manera distinta.

En una ColaLD, además de los nodos, la estructura mantiene dos referencias: una al primero y otra al último. Ese pequeño costo adicional permite mejorar el costo temporal de Acolar y Desacolar, que pasan a ser constantes. Este ejemplo muestra que muchas veces existe una relación entre tiempo y espacio: se puede usar más información interna para mejorar el tiempo de una operación.

En el caso de conjuntos y diccionarios, la estructura dinámica mejora la flexibilidad de memoria, pero no necesariamente mejora el costo temporal. Esto ocurre porque el problema principal sigue siendo la búsqueda secuencial. Es decir, aunque los nodos se

creen dinámicamente, si para encontrar un elemento hay que recorrer la estructura, el costo temporal seguirá siendo lineal.

Por lo tanto, el costo espacial ayuda a tomar mejores decisiones de diseño. Una implementación estática puede ser razonable si el tamaño máximo es conocido y no se espera superar esa capacidad. Una implementación dinámica puede ser más adecuada si el tamaño es variable, si no se conoce la cantidad máxima de elementos o si se quiere evitar reservar memoria innecesaria desde el inicio.

La idea central es que **la eficiencia no se mide solamente por velocidad**. Una implementación puede ser rápida pero reservar demasiada memoria, o puede usar memoria de manera flexible pero tener mayor sobrecarga por nodo. En esta materia, el análisis principal seguirá siendo temporal, pero el costo espacial debe ser considerado cuando se comparan implementaciones estáticas y dinámicas.